

要使用的数据全部加锁，因此遵守两段锁协议的事务可能发生死锁，如图 12.10 所示。

事务 T_1	事务 T_2
SLOCK B $R(B)=2$	
	SLOCK A $R(A)=2$
XLOCK A 等待 等待	XLOCK B 等待

图 12.10 遵守两段锁协议的事务可能发生死锁

12.8 封锁的粒度

封锁对象的大小称为**封锁粒度**(lock granularity)。封锁对象可以是逻辑单元，也可以是物理单元。以关系数据库为例，封锁对象可以是这样一些逻辑单元：属性值，属性值的集合，元组，关系，索引项，整个索引直至整个数据库；也可以是这样一些物理单元：页(数据页或索引页)，物理记录等。

封锁粒度与系统的并发度和并发控制的开销密切相关。直观地看，封锁的粒度越大，数据库所能够封锁的数据单元就越少，并发度就越小，系统开销也越小；反之，封锁的粒度越小，并发度越高，但系统开销也就越大。

例如，若封锁粒度是数据页，事务 T_1 需要修改元组 L_1 ，则 T_1 必须对包含 L_1 的整个数据页 A 加锁。如果 T_1 对 A 加锁后事务 T_2 要修改 A 中的元组 L_2 ，则 T_2 被迫等待，直到 T_1 释放 A 上的锁。如果封锁粒度是元组，则 T_1 和 T_2 可以分别对 L_1 和 L_2 加锁，不需要互相等待，从而提高了系统的并行度。又如，事务 T 需要读取整个表，若封锁粒度是元组， T 必须对表中的每一个元组加锁，显然开销极大。

因此，在一个系统中如能同时支持多种封锁粒度供不同的事务选择是比较理想的，这种封锁方法称为**多粒度封锁**。选择封锁粒度时应同时考虑封锁开销和并发度两个因素，适当选择封锁粒度以求得最优的效果。一般说来，需要处理某个关系的大量元组的事务，可以以关系为封锁粒度；需要处理多个关系的大量元组的事务，可以以数据库为封锁粒度；而对于一个处理少量元组的用户事务，则以元组为封锁粒度就比较合适。

12.8.1 多粒度封锁

首先定义**多粒度树**。多粒度树的根结点是整个数据库，表示最大的数据粒度。叶结点表示

最小的数据粒度。

图 12.11 给出了一个三级粒度树。根结点为数据库，数据库的子结点为关系，关系的子结点为元组。



图 12.11 三级粒度树

多粒度封锁协议允许多粒度树中的每个结点被独立地加锁。对一个结点加锁，意味着这个结点的所有后裔结点也被加以同样类型的锁。因此，在多粒度封锁中一个数据对象可能以两种方式封锁，显式封锁和隐式封锁。

显式封锁是指应事务的要求直接加到数据对象上的锁。**隐式封锁**是指该数据对象没有被独立加锁，是因其上级结点加锁而使该数据对象加上了锁。

多粒度封锁方法中，显式封锁和隐式封锁的效果是一样的。因此，系统检查封锁冲突时不仅要检查显式封锁，还要检查隐式封锁。例如，事务 T 要对关系 R_1 加 X 锁，系统必须搜索其上级结点数据库、关系 R_1 以及 R_1 的下级结点，即 R_1 中的每一个元组，上下搜索。如果其中某一个数据对象已经加了不相容锁，则 T 必须等待。

一般地，对某个数据对象加锁，系统先要检查该数据对象上是否有显式封锁与之冲突，再检查其所有上级结点，看本事务的显式封锁是否与该数据对象上的隐式封锁（即由于上级结点已加的封锁造成的）冲突；还要检查其所有下级结点，看它们的显式封锁是否与本事务的隐式封锁（将加到下级结点的封锁）冲突。显然，这样的检查方法效率很低。为此人们引进了一种新型锁，称为**意向锁**（intention lock）。有了意向锁，数据库管理系统就无须逐个检查下一级结点的显式封锁了。

12.8.2 意向锁

如果对一个结点加意向锁，则说明该结点的后裔结点正在被加锁；对任一结点加锁时，必须先对它的上层结点加意向锁。

例如，对任一元组加锁时，必须先对它所在的数据库和关系加意向锁。

下面介绍三种常用的意向锁：意向共享型锁（intent shared lock，简称 IS 锁）、意向排他型锁（intent exclusive lock，简称 IX 锁）和共享意向排他型锁（shared and intention exclusive lock，简称 SIX 锁）。

① **IS 锁**。如果对一个数据对象加 IS 锁，表示它的后裔结点拟（意向）加 S 锁。例如，事务

T_i 要对 R_i 中某个元组加 S 锁, 则要首先对关系 R_i 和数据库加 IS 锁。

② **IX 锁**。如果对一个数据对象加 IX 锁, 表示它的后裔结点拟(意向)加 X 锁。例如, 事务 T_i 要对 R_i 中某个元组加 X 锁, 则要首先对关系 R_i 和数据库加 IX 锁。

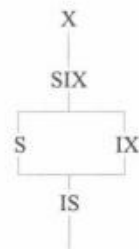
③ **SIX 锁**。如果对一个数据对象加 SIX 锁, 表示对它加 S 锁, 再加 IX 锁, 即 $SIX = S + IX$ 。例如对某个表加 SIX 锁, 则表示该事务要读整个表(所以要对该表加 S 锁), 同时会更新个别元组(所以要对该表加 IX 锁)。

图 12.12(a) 给出了这些锁的相容矩阵, 从中可以发现这 5 种锁的强度有如图 12.12(b) 所示的偏序关系。所谓锁的强度是指它对其他锁的排斥程度。一个事务在申请封锁时以强锁代替弱锁是安全的, 反之则不然。

T_i	T_j					
	S	X	IS	IX	SIX	—
S	Y	N	Y	N	N	Y
X	N	N	N	N	N	Y
IS	Y	N	Y	Y	Y	Y
IX	N	N	Y	Y	N	Y
SIX	N	N	Y	N	N	Y
—	Y	Y	Y	Y	Y	Y

注: Y=Yes, 表示相容的请求; N=No, 表示不相容的请求

(a) 封锁类型的相容矩阵



(b) 锁的强度偏序关系

图 12.12 加意向锁后锁的相容矩阵与偏序关系

在具有意向锁的多粒度封锁方法中, 任意事务 T 要对一个数据对象加锁, 必须先对它的上层结点加意向锁。申请封锁时应按自上而下的次序进行, 释放封锁时则应按自下而上的次序进行。

例如, 事务 T_i 要对关系 R_i 加 S 锁, 则要首先对数据库加 IS 锁。检查数据库是否已加了不相容的锁(X 锁), 然后检查 R_i 是否已加了不相容的锁(X 或 IX 锁), 不再需要搜索和检查 R_i 中的各元组是否加了不相容的锁(X 锁)。

具有意向锁的多粒度封锁方法提高了系统的并发度, 减少了加锁和解锁的开销, 已在实际的数据库管理系统产品中得到广泛应用。

* 12.9 其他并发控制机制

并发控制的方法除了封锁技术外还有时间戳方法、乐观方法和多版本并发控制方法等。这里做一个概要的介绍。

时间戳方法给每一个事务“盖”上一个时标, 即事务开始执行的时间。每个事务具有唯一的时间戳, 并按照这个时间戳来解决事务的冲突操作。如果发生冲突操作, 就回滚具有较早时